

JAVA SDE-2 INTERVIEW PREPARATION SERIES

FRAMEWORKS, DATA, AND MESSAGING - BOOK 11 OF 12 - STUDY STEP FW-11

Spring Ecosystem Extensions

Security, Cloud, WebFlux, Batch, Integration, Actuator, and Test Strategy

BY

Vinay Reddy Kalluri

A problem-first guide to Security, Cloud, observability, WebFlux, Batch, Integration, testing, and their production failure boundaries.

SECURITY | CLOUD | WEBFLUX | BATCH | ACTUATOR | TESTING

JAVA 21 | INTERVIEW CORE | SDE-2 FOLLOW-UPS | PRINTABLE EDITION

Series edition - Updated 2026-08-02

Open educational interview-preparation edition

Start Here - Your Route Through This Volume

60-second entry rule: If two or more recognition signals below expose a gap, begin with Chapter 1. If the prerequisites are weak, step back to FW 10 – Apache Kafka and Spring Kafka. If you can already explain every outcome without notes, jump to Part II and prove it through the practice ladder.

Teach the major Spring extensions from prerequisites and runtime mechanics through SDE-2 trade-offs without implying every service needs every module.

Readiness map

Decision	Evidence
START HERE	Authentication, distributed configuration, reactive I/O, batch work, or integration flows enter scope; Operational endpoints and test strategy need explicit design; A specialized Spring module may solve a well-defined boundary.
PREPARE FIRST	Spring Framework, Boot, and Data books; Kafka and Redis basics recommended.
MOVE ON WHEN	Trace authentication, authorization, reactive, batch, integration, and observability runtime boundaries; Design bounded deadlines, retries, idempotency, restart, and operational evidence; Choose the smallest Spring extension justified by a system requirement.

Choose the route that fits today

- 1 **Learning:** read in order, type the representative Java examples, and complete each dry run.
- 2 **Revision or rehearsal:** start at the first decision map, invariant, or failure mode you cannot explain; if none expose a gap, go directly to Part II and prove readiness without notes.

Frameworks Segment Position

FW 10 - Apache Kafka and Spring Kafka	YOU ARE HERE FW 11 - Spring Ecosystem Extensions	FW 12 - Spring AI
---------------------------------------	---	-------------------

A note from Vinay

The Spring ecosystem is broad, so this book teaches selection judgment rather than annotation collecting. Add an extension only when its lifecycle and failure semantics fit the problem you actually have.

Contents

This contents page covers only the current focused PDF. Section-level navigation is available through PDF bookmarks.

CONTENTS NAVIGATION	
Start Here - Your Route Through This Volume	2
Part I - Core Learning	4
Chapter 1 - Spring Ecosystem Extensions: Start with the Problem, Not the Starter	4
Chapter 2 - Spring Security: From HTTP Request to an Authorized Method	9
Chapter 3 - Actuator, Observability, and Tests That Prove the Runtime	15
Chapter 4 - Spring Cloud: Configuration, Clients, Gateway, and Resilience Budgets	19
Chapter 5 - Spring WebFlux: Demand, Event Loops, and Blocking Boundaries	24
Chapter 6 - Spring Batch and Spring Integration: Durable Progress and Message Flow	29
Chapter 7 - Cross-Module Failures and the Smallest-Module Decision	34
Chapter 8 - Realistic Spring Ecosystem Interview Rounds	38
Chapter 9 - Practice, Reasoned Solutions, and Readiness	43
Chapter 10 - Java 21 Spring Ecosystem Runtime Companion	47
Part II - Practice and Handoff	59

Part I - Core Learning**SDE-2 CORE CHAPTER**

Chapter 1 - Spring Ecosystem Extensions: Start with the Problem, Not the Starter

Spring has a module for almost every backend concern. That is useful, but it creates a common interview mistake: naming a starter before explaining the problem. An SDE-2 answer should travel in the opposite direction:

TEXT

```
business invariant
  |
  v
runtime and failure boundary
  |
  v
smallest useful abstraction
  |
  v
Spring module + configuration + proof
```

This book follows that order. It assumes that you already understand plain Java, dependency injection, Spring proxies, transactions, Spring Boot configuration, HTTP, and basic persistence. It does **not** assume that you know Spring Security, Cloud, WebFlux, Batch, or Integration.

Version boundary

The examples target the Spring Boot 4.1 generation and Java 21. At publication time, that line aligns with Spring Framework 7, Spring Security 7, Spring Batch 6, and the Spring Cloud 2025.1 release train. Spring projects ship independently, so treat exact annotations, defaults, and artifact names as version-sensitive.

The durable ideas are older than any release:

- authentication establishes an identity; authorization decides what that identity may do;
- a request is transformed by an ordered chain before application code runs;
- reactive code needs an end-to-end non-blocking path to gain its scalability model;
- a restartable batch job must record progress at a safe boundary;
- integration flows move messages through explicit channels and endpoints;
- retries are safe only when the operation and the failure are classified;
- health, logs, metrics, and traces answer different questions.

Use a Bill of Materials rather than choosing Spring Cloud component versions independently. For Boot 4.0/4.1, the Spring project compatibility table places applications on the 2025.1 release train; re-check the official table before an upgrade.

What belongs in this volume

Requirement	First module to investigate	What it does not solve by itself
Identity, request protection, permissions	Spring Security	User lifecycle, business authorization policy, secret storage
Health, metrics, traces, operational controls	Boot Actuator + Micrometer	A useful SLO, incident response, safe endpoint exposure
External configuration, gateway, service clients	Spring Cloud projects	Distributed consistency, correct timeout budgets, good service boundaries
High-concurrency non-blocking I/O	Spring WebFlux + Reactor	Slow blocking dependencies, CPU capacity, simpler MVC workloads
Restartable finite data processing	Spring Batch	Continuous event processing, idempotent business semantics
Enterprise application integration flows	Spring Integration	Broker durability unless the selected channel/adaptor provides it
Focused framework proof	Spring Test + provider fixtures	Production-equivalent behavior without realistic dependencies

Do not add all of these to a service "for future use." Every module increases configuration surface, upgrade work, test combinations, and operational knowledge.

One running system

The chapters use an order platform:

TEXT
<pre> browser / API client v gateway -> order API -> database -> payment API -> audit event nightly settlement file -> batch job -> reconciliation table partner SFTP -> integration flow -> validated message -> order API </pre>

The important questions are concrete:

- 1 Who authenticated the caller, and where is that identity stored?
- 2 Which component authorized **order:refund** for this order and tenant?

- 3 Which timeout expires first when payment is slow?
- 4 Does a retry repeat a side effect?
- 5 Which thread executes each stage, and can it block?
- 6 What state lets a failed import resume without duplicating rows?
- 7 Which signal tells an operator that the system is wrong rather than merely busy?

Learning sequence

TEXT

```

module selection and prerequisites
    |
    v
Security request path -> authentication -> authorization -> exploit defenses
    |
    v
Actuator, metrics, traces, and focused tests
    |
    v
Cloud configuration, clients, gateway, and resilience
    |
    v
WebFlux demand, event loops, and blocking boundaries
    |
    v
Batch checkpoints + Integration message flows
    |
    v
failure matrix -> live interview rounds -> exercises and solutions
  
```

Security comes first because every later module must preserve identity and tenant context. Observability comes before distributed resilience because you cannot tune what you cannot see. WebFlux follows ordinary request flow so "reactive" has something familiar to compare against.

The boundary tracing habit

For any extension, draw five labels:

TEXT

```

caller -> framework boundary -> application boundary -> external resource
      |               |               |
      context         invariant
      |
      thread / transaction / retry attempt
  
```

Then ask:

- What enters and leaves the boundary?
- Which state is thread-local, subscriber-context, persisted, or reconstructed?

- Which order is guaranteed and which order is incidental?
- What happens on timeout, cancellation, duplicate delivery, restart, and shutdown?
- Which test or telemetry proves the behavior?

A small decision example

Requirement: Import a 20 GB settlement file once per night, resume after a crash, and skip already committed records.

Weak answer: "Use WebFlux because it handles a lot of data."

Stronger reasoning:

- 1 The workload is finite and restartability matters more than keeping many sockets open.
- 2 Records can be read in bounded chunks.
- 3 A durable job repository can record which step and checkpoint committed.
- 4 The writer needs an idempotency key because a crash can occur after an external effect but before metadata advances.
- 5 Spring Batch is the first module to evaluate. WebFlux is not the natural abstraction for job identity or restart state.

Quick check

- 1 Why is a Spring starter not a design justification?
- 2 When is WebFlux a poor choice even if throughput is important?
- 3 What is the difference between batch restartability and business idempotency?
- 4 Why must a Spring Cloud release train be aligned with Boot?
- 5 What context must survive an asynchronous boundary?

Practice

- **Foundation:** Match each running-system requirement to Security, Actuator, Cloud, WebFlux, Batch, Integration, or no extra module.
- **Interview Core:** Draw the five labels for an authenticated call from gateway to order service to payment service.
- **Interview Core:** Explain why adding retries without an idempotency design can make availability worse.
- **SDE-2 Follow-up:** Choose between MVC, WebFlux, Batch, and messaging for three workloads and state the evidence that could change your choice.

TRY IT | Readiness checkpoint

Continue when you can describe each extension by the runtime problem it solves, one cost it introduces, and one failure it cannot solve on its own.

SDE-2 CORE CHAPTER

Chapter 2 - Spring Security: From HTTP Request to an Authorized Method

Security is not "add login." It is a pipeline that establishes trustworthy context and then applies policy at every sensitive boundary.

Four ideas before configuration

Idea	Plain-language meaning	Typical Spring Security representation
Principal	Who the caller claims to be	<code>Authentication.getPrincipal()</code>
Credential	Evidence used to prove the claim	Password, session, bearer token, mTLS evidence
Authentication	The result of verifying identity evidence	An authenticated <code>Authentication</code>
Authorization	A decision about an action on a resource	Request rules, <code>AuthorizationManager</code> , method security

Roles and authorities are inputs to policy, not the policy itself. "Has `ADMIN`" may be enough for an internal tool; "may refund this order for this tenant up to this amount" needs resource-aware business rules.

Servlet request flow

Spring Security's servlet support is a set of filters behind a `DelegatingFilterProxy`. The proxy delegates into Spring-managed `FilterChainProxy`, which selects the first matching `SecurityFilterChain`.

TEXT

```
HTTP request
|
container filter chain
|
DelegatingFilterProxy
|
FilterChainProxy
|
first matching SecurityFilterChain
|
load SecurityContext
-> exploit protection
-> authentication filter(s)
-> exception translation
-> authorization
|
controller -> service -> repository
```

Two ordering rules explain many bugs:

- 1 Only the **first matching** security chain is selected. A broad matcher placed first can shadow a specific API chain.
- 2 Authentication must happen before a rule that needs the authenticated principal.

Do not memorize every built-in filter position. Learn the events: context loading, protection, authentication, exception translation, authorization.

Minimal version-labeled configuration

For the Spring Security 7 generation, explicit lambdas make intent visible:

JAVA

```
@Bean
SecurityFilterChain apiSecurity(HttpSecurity http) throws Exception {
    return http
        .securityMatcher("/api/**")
        .authorizeHttpRequests(authorize -> authorize
            .requestMatchers("/api/public/**").permitAll()
            .requestMatchers(HttpMethod.POST, "/api/orders/*/refund")
                .hasAuthority("order:refund")
            .anyRequest().authenticated())
        .oauth2ResourceServer(resourceServer -> resourceServer.jwt(Customizer.withDefaults()))
        .build();
}
```

This is intentionally not a complete production policy. It does not decide whether the caller owns a particular order, whether the tenant matches, or whether the amount exceeds an approval limit.

Authentication internals

A username/password flow makes the contracts easy to see:

TEXT

```
filter extracts credentials
    |
    v
AuthenticationManager
    |
    v
ProviderManager tries compatible AuthenticationProvider(s)
    |
    +--> user lookup / credential verification
    |
    v
authenticated Authentication
    |
    SecurityContextHolder for the current request
```

The raw password should be verified by a one-way adaptive password encoder. It should not be decrypted. On successful login, change-sensitive operations may require session fixation protection and CSRF protection depending on the client model.

For a bearer-token resource server, the service normally validates signature, issuer, audience, time claims, and maps claims to authorities. A signed token is not automatically acceptable: it can be issued by the wrong issuer, intended for another audience, expired, not-yet-valid, or missing the tenant claim needed by the application.

Session, bearer token, and context lifetime

Model	Identity state	Main risk
Server session	Session identifier points to server-held security context	Session theft/fixation, distributed session consistency
Self-contained bearer token	Each request carries signed claims	Revocation delay, incorrect audience/issuer validation, claim over-trust
Opaque token	Resource server introspects or validates through an authority	Authority dependency and latency
mTLS plus application identity	Transport proves a certificate; app maps it to policy	Certificate lifecycle and confusing transport identity with user intent

`SecurityContextHolder` is traditionally thread-bound in servlet applications. Starting asynchronous work does not mean the context safely follows. Prefer passing the minimum immutable identity/tenant data needed by the use case, or use a framework-supported context propagation strategy and test it. Never assume arbitrary executor threads inherit request state.

Authorization at two boundaries

Request authorization rejects obviously forbidden routes early. Method authorization protects application operations even when they acquire a second caller later.

JAVA

```
@PreAuthorize("hasAuthority('order:refund')")
public RefundReceipt refund(OrderId orderId, Money amount, Actor actor) {
    Order order = repository.require(orderId);
    policy.requireRefundAllowed(actor, order, amount);
    return refundProcessor.refund(order, amount);
}
```

The annotation checks a coarse capability. `policy.requireRefundAllowed` evaluates object ownership, tenant, state, and monetary rules. That domain decision should be testable without starting Spring.

Method security commonly uses a proxy. The familiar proxy boundaries still apply: the object must be managed, infrastructure must be enabled, and self-invocation may bypass interception. Do not rely on one annotation as the only protection around a privileged external side effect.

CSRF, CORS, and headers are different concerns

CSRF exploits a browser automatically attaching credentials such as cookies. If a browser authenticates with a session cookie, state-changing requests generally need CSRF protection. A stateless API that accepts only an **Authorization** bearer header does not receive that header automatically from a hostile site, but disabling CSRF must follow the actual credential model rather than the word "REST."

CORS is a browser rule controlling which origins may read or send selected cross-origin requests. It is not authentication and does not stop curl or another server.

Security headers reduce browser attack surface, but they do not sanitize business data or replace output encoding and content-security design.

OAuth 2.0 and OpenID Connect without role confusion

- OAuth 2.0 delegates authorization to access protected resources.
- OpenID Connect adds an identity layer and ID tokens for client login.
- A resource server validates access tokens.
- A client obtains/uses tokens to call a resource server.
- An authorization server issues tokens after the relevant grant and user/client checks.

Do not send an ID token to an API as if it were an access token. Do not implement an authorization server inside each business service. Use Authorization Code with PKCE for public/browser-native clients; avoid obsolete password-based grants.

Failure and edge-case matrix

Scenario	Naive expectation	Actual risk	Strong response
Broad chain matcher before narrow chain	Both chains combine	First matching chain wins	Order and test representative paths
Valid JWT with wrong audience	Signature means trusted	Token was minted for another API	Validate issuer, audience, times, algorithm policy
Method calls another secured method on <code>this</code>	Both annotations run	Self-invocation may bypass proxy	Move boundary, call through collaborator, test denied path
Async task reads current user	Context follows work	Thread-local context is absent/stale	Pass explicit actor data or proven propagation
CORS allows origin	Caller is authorized	CORS is only browser access policy	Authenticate and authorize independently
CSRF disabled for cookie login	API is "stateless"	Browser still sends cookie automatically	Preserve token-based CSRF defense or redesign credentials
<code>permitAll</code> endpoint logs raw token	Public route is harmless	Secret leaks to logs	Redact headers and payloads by policy
<code>ROLE_ADMIN</code> across tenants	Admin can manage all	Cross-tenant data exposure	Combine authority with tenant/resource checks
Cached permission after revocation	Cache improves latency	Stale authorization window	Define TTL/invalidation and critical-operation recheck

Debugging sequence

When an endpoint unexpectedly returns 401 or 403:

- 1 Confirm which `SecurityFilterChain` matched.
- 2 Print or inspect the filter chain in a safe environment.
- 3 Distinguish authentication failure (often 401) from an authenticated-but-denied decision (often 403).
- 4 Inspect issuer, audience, expiry, clock skew, and authority mapping-not the raw token in shared logs.
- 5 Confirm request matcher semantics and HTTP method.
- 6 Confirm method-security proxy crossing and resource policy.
- 7 Add one allowed and one denied integration test.

Quick check

- 1 Why can a valid signature still produce an unacceptable JWT?
- 2 What chooses among multiple servlet security chains?
- 3 Why is CORS not access control?
- 4 Where should tenant-aware authorization live?
- 5 Why can async execution lose the current security context?

Practice

- **Foundation:** Draw authentication and authorization as separate steps for session login.
- **Interview Core:** Diagnose why `/api/admin/**` is public when two filter chains exist.
- **Interview Core:** Write a policy interface for tenant-aware order refunds and test it in plain Java.
- **SDE-2 Follow-up:** Design revocation behavior for a 15-minute bearer token used by a high-risk payment endpoint.

SDE-2 CORE CHAPTER

Chapter 3 - Actuator, Observability, and Tests That Prove the Runtime

Operations are part of the design. A service is not production-ready because `/actuator/health` says UP; it is ready when its signals help a human decide what is failing, who is affected, and what action is safe.

Logs, metrics, traces, and profiles

Signal	Best question	Common misuse
Logs	What discrete event happened with which safe context?	Treating every request as a verbose string dump
Metrics	How often, how much, how long, how many?	User/order IDs as labels causing cardinality explosion
Traces	Where did one request spend time across boundaries?	Sampling away every rare failure or recording secrets
Profiles/JFR	Where is CPU, allocation, lock, or runtime pressure?	Profiling only after guessing a code line

Actuator exposes endpoints and contributes health/metrics integration. Micrometer supplies instrumentation abstractions. A tracing backend stores and queries spans. None of these chooses an SLO for you.

Request and observation flow

TEXT

```
HTTP request
|
server observation starts
|  trace/span context
v
controller -> service -> database/client instrumentation
|               |               |
|               domain metric   child span / timer
v
response status + latency recorded
|
exporter -> metrics/tracing backend
```

Context propagation is part of correctness. If work crosses an executor or reactive boundary without its observation context, traces fragment. If you copy all request state blindly, secrets can leak. Propagate supported tracing context and explicit business correlation identifiers; do not invent an unbounded thread-local bag.

Health is a contract, not a dependency census

Use distinct probes when the platform supports them:

- **Liveness:** should this process be restarted? It should not fail merely because a remote dependency is temporarily down.
- **Readiness:** should new traffic be routed here? It may account for local initialization and dependencies essential to serving the route.
- **Deep diagnostic health:** can an operator inspect dependency detail? Keep this authenticated and separate from orchestration probes.

If liveness calls the database and the database fails, every instance may restart at once, adding load and erasing useful evidence. If readiness performs ten slow network calls, probes can become an outage amplifier.

Safe endpoint exposure

Actuator endpoints can reveal environment values, request mappings, log levels, heap information, or configuration structure. Apply least exposure:

TEXT

```
public traffic network: business endpoints only
management network: selected health/metrics endpoints
authenticated operator path: sensitive diagnostics
```

Sanitize secrets, constrain who may mutate log levels or shutdown behavior, and do not assume an obscure path is a security control.

Useful SDE-2 measurements

For the order path, start with service-level evidence:

- request rate, error rate, and latency distribution;
- dependency latency and outcome by bounded low-cardinality dimensions;
- executor active count, queue depth, rejections;
- connection-pool acquisition wait and usage;
- retry attempts and exhausted retries;
- circuit state changes, but not only circuit state;
- batch lag, processed/failed/skipped records, restart count;
- authorization denials by policy family, without user ID labels.

A percentile is not an individual trace. A timer around a retry wrapper may hide that one logical request made four remote attempts. Record logical request outcome and attempt-level evidence intentionally.

Test pyramid for extensions

TEXT

```

many: pure Java policy and transformation tests
|
focused: security filter/method, reactive publisher, batch step tests
|
provider: database/broker/identity/vector-store contract tests
|
few: full application journeys under realistic failure

```

What each test should prove

Concern	Fast proof	Boundary proof
Authorization	Policy allows/denies resource cases	Mock user/JWT crosses actual filter and method boundary
Reactive flow	Publisher respects demand/cancellation	Real non-blocking client; blocking detector where used
Batch restart	Processor/writer invariants	Fail after checkpoint, restart with job repository
Integration flow	Transformer/router behavior	Channel/adaptor error and redelivery path
Cloud client	Timeout/retry classifier in plain Java	Stub server produces latency/reset/status sequence
Observability	Tag policy and outcome mapping	Registry/tracing test sees expected low-cardinality signal

Starting a whole context for a pure authorization predicate makes the test slower without proving more. Mocking an entire HTTP client cannot prove timeout configuration or wire semantics.

Failure and edge-case matrix

Failure	Misleading signal	Better evidence/action
Database down	Liveness fails and pods restart	Keep process live; readiness/route health and dependency metric degrade
Thread pool saturated	CPU looks low	Queue depth, active threads, rejection count, request latency
Retry storm	Dependency call success eventually	Attempts per logical request, total load, timeout budget
High-cardinality metric	Dashboard initially looks detailed	Bound labels; put unique IDs in sampled logs/traces
Trace missing async stage	Parent request looks fast	Supported context propagation and explicit async span
Health endpoint public	Convenience	Separate management plane and authorize detail
Test passes with mock	Production TLS/DNS/pool fails	Add provider/wire-level fixture
Global context dirtied per test	"Isolation"	Slow suite and hidden shared design

Quick check

- 1 Why should a database failure not normally fail liveness?
- 2 Why is `userId` a dangerous metric label?
- 3 What evidence distinguishes pool exhaustion from slow SQL?
- 4 Which test proves a Spring Security filter chain actually matched?
- 5 Why can retry metrics be misleading when recorded only around the outer call?

Practice

- **Foundation:** Classify five signals as log, metric, trace, or profile.
- **Interview Core:** Design liveness and readiness for an API that can serve cached reads while its database is down.
- **Interview Core:** Replace a metric labeled by order ID with a safe metric plus trace/log context.
- **SDE-2 Follow-up:** Build a failure-injection test plan for timeout, retry, circuit breaker, and fallback behavior.

SDE-2 CORE CHAPTER

Chapter 4 - Spring Cloud: Configuration, Clients, Gateway, and Resilience Budgets

Spring Cloud is a family of projects, not a distributed-systems cure. Use it when a deployment genuinely needs shared configuration, service discovery, a gateway, managed service clients, or integration with a platform's distributed capabilities.

Start with topology

Before choosing a module, draw the call path:

TEXT

```
mobile client
  |
edge / load balancer
  |
API gateway
  |
order-service --HTTP--> payment-service
  |                   |
database              provider API
```

Mark who owns TLS termination, authentication, routing, service discovery, timeout, retry, load balancing, and observability. Duplicate ownership is a common production defect: a platform mesh retries, the gateway retries, and the client retries the same call.

Release trains and dependency management

Spring Cloud components are released as a tested train. Import the official `spring-cloud-dependencies` BOM compatible with the chosen Spring Boot generation. Do not select individual transitive versions because one library appears newer.

For the Boot 4.0/4.1 generation, the official compatibility table maps to Spring Cloud 2025.1. Exact service releases change; verify the table and use the latest supported service release for your line.

External configuration flow

TEXT

```
configuration source
  |
  v
config client bootstrap/import
  |
PropertySources ordered by precedence
  |
binding + validation
  |
immutable configuration object
```

Important edge cases:

- A remote source can be unavailable during startup. Decide whether the service must fail closed or may start from safe local defaults.
- Refreshable configuration can change behavior while requests are running. Not every property is safe to refresh.
- Precedence can let an environment variable override a remote value-or the reverse-depending on configuration.
- Secrets need a secret manager and access controls, not a public config repository.
- Configuration must be validated as a coherent set. A new timeout with an old retry count may violate the total request budget.

Prefer immutable, validated configuration properties over scattered string lookups.

Discovery and load balancing

Discovery maps a logical service name to eligible instances. Client-side load balancing chooses one instance; platform DNS or a service mesh may already perform this role.

Ask:

- 1 How quickly does a new healthy instance appear?
- 2 How quickly is a failed instance removed?
- 3 Is the view stale or eventually consistent?
- 4 Does the client retry the same instance or choose another?
- 5 Does session affinity exist, and is the service truly stateless?

Discovery says where a service might be. It does not say the chosen instance will complete within the caller's deadline.

Gateway request flow

TEXT

```
request
-> route predicate match
-> ordered pre-filters
-> downstream HTTP exchange
-> ordered post-filters
-> response
```

A gateway is useful for coarse cross-cutting concerns: routing, TLS, authentication handoff, rate limits, request-size limits, and consistent telemetry. It should not become a hidden business workflow engine.

Filter order matters. For example, authentication must establish identity before a tenant rate limiter uses it. Body-caching filters can consume significant memory. A fallback route can turn a clear 503 into stale or incorrect business data.

Declarative clients do not remove HTTP

An interface-based client still performs DNS lookup, connection acquisition, TLS, serialization, network I/O, status mapping, and response-body consumption.

TEXT

```
service method
-> client proxy
-> request encoder/interceptors
-> HTTP client connection pool
-> network
-> response decoder/error mapping
-> return or exception
```

Define explicitly:

- connect timeout;
- connection-acquisition timeout;
- response/read timeout;
- maximum response/body size;
- error mapping;
- retry classification;
- propagation of trace and minimum safe identity context.

One end-to-end timeout budget

Suppose the API must respond within 800 ms:

TEXT		
gateway overhead	50 ms	
order service local work	100 ms	
payment call budget	500 ms	
response reserve	150 ms	

total	800 ms	

Three payment attempts of 500 ms cannot fit. A retry policy must consume the remaining deadline, not reset a fresh timeout per attempt. Add backoff and jitter so replicas do not synchronize.

Retry, circuit breaker, bulkhead, rate limit

Mechanism	Protects against	Does not guarantee
Timeout/deadline	Waiting forever	Operation did not complete remotely
Retry	A classified transient failure	Side effect was not duplicated
Circuit breaker	Repeated calls to a failing dependency	Dependency recovery or data correctness
Bulkhead/concurrency limit	One dependency consuming all local capacity	Fairness or downstream capacity
Rate limit	Excess request rate at a boundary	Backend latency or per-key business correctness
Fallback	A deliberately degraded response	Fresh or semantically equivalent data

The ambiguous timeout

The client times out after sending a payment request. The server may have:

- 1 received nothing;
- 2 committed payment and lost the response;
- 3 still been processing when the connection closed.

Blind retry is unsafe. Send a stable idempotency key and design the payment API to return the original result for a duplicate key. If the provider lacks idempotency, reconcile before repeating the effect.

Failure and edge-case matrix

Scenario	Failure mode	Design response
Config server unavailable	Startup fleet outage	Explicit fail-fast/fallback policy and cached-safe config only if justified
Partial config refresh	Incoherent timeout/retry values	Bind and validate a versioned configuration set
Stale discovery entry	Connection failure	Short bounded retry to another instance if operation is safe
Gateway and client both retry	Multiplicative attempts	Assign one retry owner and emit attempt counts
Circuit opens on caller errors	Healthy dependency hidden	Classify only relevant failures/latency
Fallback returns empty list	Outage appears as "no data"	Mark degradation or fail explicitly
Bulkhead queue unbounded	Memory and latency collapse	Bound concurrency and queue; reject early
Token forwarded unchanged	Excess privilege downstream	Audience-bound token exchange or minimal trusted claims
Client reads huge error body	Memory pressure	Bound body and map status safely

Quick check

- 1 Why can nested retries make a three-attempt policy produce nine calls?
- 2 What does service discovery not guarantee?
- 3 What state makes a payment retry safe after a timeout?
- 4 Why should fallback data be visibly degraded?
- 5 How do release trains reduce upgrade risk?

Practice

- **Foundation:** Draw one client call and label every timeout.
- **Interview Core:** Allocate a 600 ms request budget across two sequential dependencies.
- **Interview Core:** Classify connect reset, 400, 429, 503, and read timeout for retry.
- **SDE-2 Follow-up:** Design gateway and client ownership so auth, rate limiting, retry, and tracing do not execute twice.

SDE-2 CORE CHAPTER

Chapter 5 - Spring WebFlux: Demand, Event Loops, and Blocking Boundaries

WebFlux is not "MVC but faster." It is a reactive HTTP stack whose useful scaling model depends on asynchronous, non-blocking work across the entire hot path.

Begin with the Reactive Streams contract

A publisher does not push an unlimited number of items at a subscriber. The subscriber receives a subscription and requests demand.

TEXT

```
Subscriber -> subscribe -> Publisher
Subscriber <- onSubscribe(Subscription)
Subscriber -> request(n)
Subscriber <- onNext(item) at most n times
Subscriber <- onComplete OR onError
Subscriber -> cancel() when no longer interested
```

Backpressure is the rule that production should respect downstream demand. It is not the same as a business queue, a broker, or unlimited buffering.

- `Mono<T>` represents zero or one asynchronous result.
- `Flux<T>` represents zero to many asynchronous results.
- A publisher is usually **lazy**: constructing a pipeline does not execute it.
- Operators describe behavior; subscription activates it.
- Errors are terminal signals unless transformed into a new publisher.

WebFlux request flow

TEXT

```
socket event
|
reactive HTTP server / event-loop thread
|
WebFilter chain
|
DispatcherHandler
|
handler mapping -> controller/function
|
reactive service -> non-blocking client/driver
|
signals return as data becomes ready
```

An event loop can serve many in-flight requests because it does not dedicate a waiting thread to each socket. If application code blocks that event-loop thread, unrelated requests sharing it stop making progress.

A simple composition

JAVA

```
Mono<OrderView> findOrder(String orderId) {
    return orderRepository.findById(orderId)
        .switchIfEmpty(Mono.error(new OrderNotFoundException(orderId)))
        .flatMap(order -> paymentClient.findStatus(order.paymentId()))
        .map(status -> OrderView.from(orderId, status));
}
```

Read it as a dependency graph:

- 1 Subscribe to order lookup.
- 2 If it completes empty, produce a not-found error.
- 3 When an order arrives, start the dependent payment call.
- 4 When status arrives, map to a view.

map transforms a value synchronously. **flatMap** composes another asynchronous publisher. Returning **null** from either callback violates the expected contract.

Concurrency is not parallelism

Reactive pipelines can interleave many operations without running CPU work in parallel. Scheduler choice controls where work executes. Operator placement controls which part moves.

JAVA

```
Mono<Result> result = Mono.fromCallable(() -> blockingLegacyCall())
    .subscribeOn(Schedulers.boundedElastic())
    .timeout(Duration.ofMillis(300));
```

This isolates a small unavoidable blocking call on a bounded worker pool. It does not make the dependency non-blocking. Under load, the pool and its queue can still saturate. Measure and bound the bridge; prefer a truly asynchronous driver for sustained hot paths.

Never call `block()` in a reactive request handler. It can deadlock or exhaust event-loop capacity and discards cancellation/context advantages.

flatMap, ordering, and concurrency

JAVA

```
Flux.fromIterable(orderIds)
    .flatMap(this::loadOrder, 16)
```

This allows up to 16 inner subscriptions. Results can arrive out of input order. Use `concatMap` when sequential ordering is required, or an ordered variant when concurrency and ordering are both justified. Do not choose unbounded concurrency for a downstream service with finite capacity.

Error, timeout, retry, and cancellation

TEXT

```
source -> timeout -> retry classifier -> fallback/translation -> subscriber
```

Operator order changes semantics. Retrying outside a timeout may create a timeout per attempt. A timeout outside retry may cap the entire operation. Decide intentionally.

`onErrorResume` is not a universal "keep going." A fallback must preserve the API's business meaning. Turning payment failure into `PAID=false` may incorrectly imply an authoritative unpaid state.

Cancellation is cooperative. An HTTP client may cancel an in-flight exchange, but an already committed external side effect is not undone. Cleanup belongs in resource-aware operators and the underlying client/driver contract.

Reactor context and security/trace context

Thread-local reasoning is unreliable because signals can execute on different threads. Reactor **Context** is associated with a subscription, not a mutable global map. Framework integrations can propagate security and observation context through supported operators.

Keep tenant/authorization decisions explicit. A value captured before a context switch is clearer than reading an ambient value deep inside business logic.

Hot, cold, and accidental duplicate work

A cold publisher commonly performs its work per subscription. Two subscriptions can cause two HTTP calls:

JAVA

```
Mono<Quote> quote = quoteClient.fetch();
quote.subscribe(audit::record);
quote.subscribe(response::send); // potentially a second fetch
```

Sharing/caching changes lifetime, replay, error, and memory semantics. Do not add `cache()` merely to stop duplicates; decide who owns the result and for how long.

Failure and edge-case matrix

Scenario	What surprises candidates	Better design
JDBC call inside handler	Return type is reactive, work still blocks	Use MVC for blocking stack or isolate/migrate explicitly
<code>block()</code> on event loop	Thread waits for work that needs same capacity	Compose publishers end to end
Unbounded <code>flatMap</code>	Downstream and memory flood	Set concurrency/prefetch from capacity
Two subscriptions	Source executes twice	One ownership path or deliberate share/replay policy
<code>onErrorReturn(empty)</code>	Failure becomes valid empty data	Preserve degradation/error semantics
Timeout followed by retry	Side effect may duplicate	Idempotency plus remaining-deadline policy
<code>publishOn</code> placed late	Earlier blocking stage stays on event loop	Move/isolate the source correctly
Client disconnect	Server work assumed cancelled/rolled back	Observe cancellation; side effects need own consistency
Infinite stream with fast producer	Buffer grows despite reactive API	Demand-aware source and bounded policy
Shared mutable accumulator	Interleaved subscribers corrupt state	Per-subscription immutable/reduced state

MVC or WebFlux?

Choose MVC when the dependency stack is blocking, concurrency is moderate, and the team benefits from straightforward thread-per-request reasoning. Choose WebFlux when the hot path is genuinely non-blocking, many requests spend time waiting on I/O, streaming/backpressure is required, and the team can test reactive behavior.

Virtual threads may improve the scalability of blocking code, but they do not add backpressure, make remote calls cheaper, or remove database connection limits. Compare with measurements, not ideology.

Quick check

- 1 What activates a cold reactive pipeline?
- 2 Why is `flatMap` not ordered by default?
- 3 What is the difference between moving a blocking call and making it non-blocking?
- 4 Can cancellation undo a committed remote side effect?
- 5 Why can a second subscription cause a second network call?

Practice

- **Foundation:** Trace signals for a `Mono` that completes empty.
- **Interview Core:** Rewrite nested subscriptions as one composed pipeline.
- **Interview Core:** Place timeout and retry to enforce one total 700 ms deadline.
- **SDE-2 Follow-up:** Design and measure a bounded bridge to one blocking legacy SDK.

SDE-2 CORE CHAPTER

Chapter 6 - Spring Batch and Spring Integration: Durable Progress and Message Flow

Both modules coordinate work, but they model different work.

- Spring Batch models a finite **job** made of steps with durable execution metadata and restart semantics.
- Spring Integration models a **message flow** through channels, endpoints, transformers, routers, filters, and adapters.

They can be used together; they should not be treated as synonyms.

Spring Batch vocabulary

TEXT

```
Job + identifying JobParameters -> JobInstance
one launch/restart attempt      -> JobExecution
one step attempt                -> StepExecution
restart data                    -> ExecutionContext
```

A job instance is the logical run, such as `settlementDate=2026-08-01`. A job execution is one attempt. Reusing the wrong identifying parameters can either refuse a rerun or accidentally create a second logical job.

Chunk-oriented processing

TEXT

```
begin transaction
-> read item
-> process item
-> repeat until chunk full/end
-> write chunk
-> update checkpoint/job metadata
commit transaction
```

If the transaction rolls back, the chunk may be read and processed again. That means processors should be deterministic and writers must tolerate the configured retry/restart semantics.

A chunk size of 1 maximizes checkpoint frequency but adds transaction overhead. A huge chunk reduces commits but increases rollback work, memory, and lock duration. Choose from record cost, resource limits, restart target, and database behavior.

Restartability is not idempotency

Spring Batch can remember a committed checkpoint. It cannot make an external email, HTTP charge, or non-transactional file append atomic with job metadata.

TEXT

```
external side effect succeeds
      |
process crashes before checkpoint commit
      |
restart repeats the item
```

Use a business idempotency key, transactional outbox, staging table, or reconciliation workflow. "The job is restartable" is not enough.

Skip and retry require classification

- **Retry** when the same item may succeed later within a bounded attempt/time budget: transient lock timeout, short network reset for an idempotent operation.
- **Skip** when policy permits recording a bad item and continuing: malformed optional row with an auditable reject file.
- **Fail** when correctness or systemic health is uncertain: schema mismatch, authorization failure, corrupted input header.

An unlimited skip policy turns data loss into a green job. Record counts and reasons; set thresholds; reconcile input, committed, rejected, and quarantined totals.

Scaling a batch job

Technique	Shape	Main correctness question
Parallel steps	Independent steps run together	Do they contend for the same state?
Multi-threaded/local chunking	Items processed concurrently in one process	Are reader/writer/state and ordering safe?
Partitioning	Manager assigns disjoint input ranges	Are partitions complete, non-overlapping, restartable?
Remote step/chunk pattern	Work crosses processes	How are duplicates, lost workers, and broker delivery handled?

Scale only after measuring read, transform, write, lock, and commit time. A faster reader can overload the writer or database.

Spring Integration runtime model

TEXT

```

inbound adapter
|
Message(payload + headers)
|
channel -> filter -> transformer -> router -> service activator
|
outbound adapter

```

- A **DirectChannel**-style subscribable channel invokes a subscriber in the sender's thread unless an executor-backed channel changes it.
- A queue/pollable channel buffers and requires a polling consumer.
- A publish-subscribe channel fans out to subscribers; delivery, error, and executor semantics depend on configuration.
- A channel adapter connects a one-way external boundary; a gateway usually models request/reply.

Headers can carry correlation, reply, sequence, tenant, and tracing information. Do not trust an inbound tenant/role header merely because it exists; establish it at an authenticated boundary.

Error flow and transaction boundary

For synchronous direct channels, an exception can return to the sender. For executor-backed or polling endpoints, failure occurs on another thread and may be published to an error channel or drive adapter-specific redelivery.

TEXT

```

poll external source
-> begin transaction (if configured)
-> receive message
-> handler chain
-> acknowledge/commit
-> error => rollback/redelivery or error flow

```

Do not assume one database transaction spans a remote broker, HTTP endpoint, and filesystem. Design idempotency and reconciliation across those boundaries.

Routing and correlation

Routers choose channels; splitters create related messages; aggregators reconstruct a group. An aggregator needs:

- a stable correlation key;
- a completion rule;
- a bounded store/expiry policy;
- behavior for duplicates and late messages;
- recovery after process restart if completion is durable.

Without expiry, incomplete groups can leak memory/storage forever.

Failure and edge-case matrix

Scenario	Failure	Strong response
Same job parameters relaunched	Completed instance refuses rerun	Deliberately choose identifying/non-identifying parameters
Crash after remote effect	Item repeats on restart	Stable idempotency key/outbox/reconciliation
Reader not restart-safe	Checkpoint resumes at wrong record	Persist sufficient state; test mid-chunk crash
Skip limit too high	Silent data loss	Threshold, reject artifact, reconciliation counts
Partitions overlap	Duplicate writes	Deterministic disjoint partition key plus uniqueness
Direct channel handler blocks	Sender thread stalls	Make boundary explicit; choose executor/queue only with capacity policy
Queue channel has no poller	Messages never handled	Configure and observe a polling consumer
Async handler throws	Caller sees success	Error channel/dead-letter/alert and durable semantics
Aggregator never completes	State grows forever	Timeout/expiry and partial-group policy
Retried integration handler	Side effect duplicates	Idempotent receiver/store and retry classifier

Quick check

- 1 Distinguish job instance from job execution.
- 2 Why can a committed external side effect repeat after batch restart?
- 3 When is skip acceptable?
- 4 Which thread executes a direct-channel subscriber?
- 5 What five policies does an aggregator need?

Practice

- **Foundation:** Model a CSV import as job, step, reader, processor, and writer.
- **Interview Core:** Choose a chunk size experiment and list the measurements.
- **Interview Core:** Draw error propagation for direct and executor-backed channels.
- **SDE-2 Follow-up:** Design a restartable, partitioned settlement import that calls a non-transactional partner API exactly as safely as the API permits.

SDE-2 CORE CHAPTER

Chapter 7 - Cross-Module Failures and the Smallest-Module Decision

Production incidents rarely respect module boundaries. A token is accepted at the gateway, context disappears on an executor, a client retry repeats a payment, and the readiness probe removes every replica. The right unit of reasoning is the end-to-end invariant.

One complete request trace

TEXT

1. gateway authenticates token and applies coarse route policy
2. order API validates its own audience and authorization
3. controller binds a narrow command
4. service checks tenant/resource invariant
5. client acquires connection with remaining deadline
6. payment endpoint processes stable idempotency key
7. response and attempt outcome become metrics/traces
8. audit is committed through a durable boundary

At each step, record:

- identity and tenant source;
- current thread/subscriber context;
- transaction/resource boundary;
- remaining deadline;
- attempt number and idempotency key;
- durable state before and after;
- observable success/failure signal.

Cross-module failure matrix

Incident	Hidden interaction	First evidence	Safe first design move
All replicas restart during database outage	Liveness includes DB	Probe history and restart count	Separate liveness from dependency readiness
Duplicate payment after slow response	Client retry + non-idempotent side effect	Logical request ID versus provider operations	Stable idempotency key and reconciliation
Authorized MVC call denied in async stage	Thread-bound security context lost	Trace/thread/context at handoff	Pass explicit actor or supported propagation
Gateway returns empty success during downstream outage	Fallback erased error semantics	Fallback counter and response contract	Explicit degraded response or fail
WebFlux latency climbs with low CPU	Blocking call on event loop	Thread dump/event-loop block evidence	Remove/isolate blocking dependency
Batch restart creates duplicate partner records	Checkpoint not atomic with remote side effect	Job execution + partner idempotency log	Outbox/idempotency/reconciliation
Integration memory grows steadily	Aggregator groups never complete	Group-store age/count	Expiry, completion, late-message policy
Security rules differ by endpoint	First chain matcher shadows later chain	Selected chain/filter list	Narrow matchers first; allowed/denied tests
Config refresh destabilizes clients	Timeout and retries changed independently	Config version and attempt duration	Versioned validated settings
Retry storm opens circuit everywhere	Retry multiplication and shared dependency	Calls per logical request	Single retry owner, jitter, concurrency cap
Trace disappears after reactive hop	Unsupported manual thread-local	Span graph and scheduler boundary	Framework context propagation
Batch reports completed with missing rows	Skip policy masks systemic parse error	Input/commit/reject reconciliation	Classified skip threshold and fail systemic errors

Module selection table

Workload	Prefer first	Reconsider when
Ordinary blocking CRUD API	Spring MVC	Streaming or very high I/O concurrency has measured value and dependencies are async
Browser login/API authorization	Spring Security	Never replace with custom filters; integrate an identity provider as needed
Platform already supplies DNS and gateway	Native platform + focused clients	Application needs a feature not safely supplied there
Finite nightly import with restart	Spring Batch	A simple small atomic task has no restart/scale need
Continuous broker events	Kafka/Rabbit listener stack	Flow requires integration adapters/routing across protocols
In-process transformation across adapters	Spring Integration	A few direct calls are clearer and no messaging semantics are needed
Basic health and metrics	Actuator/Micrometer	Always define SLO and exposure; module alone is incomplete

When not to add an extension

Do not add Spring Cloud discovery when Kubernetes service DNS already provides the required location model. Do not add WebFlux around a blocking JDBC/SDK path to make the controller signature look modern. Do not model a two-line method call as an Integration flow when it adds no routing, temporal decoupling, or adapter value. Do not use Batch for one small transaction triggered by a request.

The smaller design wins when it makes ordering, failure, ownership, and tests clearer.

SDE-2 design checklist

- 1 **Invariant:** What must remain true despite duplicate, delay, timeout, and restart?
- 2 **Ownership:** Which layer owns authentication, authorization, retry, routing, and state?
- 3 **Capacity:** What is bounded-threads, connections, queue, demand, partitions, group state?
- 4 **Time:** What is the end-to-end deadline, and who consumes it?
- 5 **Durability:** Which state survives process loss, and what can repeat?
- 6 **Security:** Which input is trusted, validated, least-privileged, and tenant-scoped?
- 7 **Evidence:** Which test and telemetry distinguish success, degradation, and silent loss?
- 8 **Version:** Which Boot/release-train/module line is supported together?

Quick check

- 1 Why does a fallback belong in the business contract?
- 2 What makes a queue "bounded" in operational terms?
- 3 Why can platform and application retries multiply?
- 4 What evidence proves no batch rows disappeared?
- 5 When is a direct Java call better than an Integration flow?

Practice

- **Interview Core:** Take one incident row and produce a five-minute diagnosis plan.
- **Interview Core:** Remove one unnecessary Spring extension from a hypothetical CRUD service and justify the simpler design.
- **SDE-2 Follow-up:** Design one capacity budget spanning gateway, WebFlux/client concurrency, database pool, and downstream limit.

SDE-2 CORE CHAPTER

Chapter 8 - Realistic Spring Ecosystem Interview Rounds

These are dialogue chains, not flash cards. Pause after each interviewer line and answer aloud. A strong SDE-2 answer clarifies the invariant, traces the runtime, names the failure window, makes a trade-off, and proposes proof.

1. Two security chains

Interviewer: `/api/admin/report` is unexpectedly accessible without authentication. Two `SecurityFilterChain` beans exist. Diagnose it.

Candidate: I would first log or inspect which chain matched that exact method and path. `FilterChainProxy` selects the first matching chain; it does not merge both. A broad `/**` chain ordered before `/api/admin/**` can shadow the admin chain. I would make matchers non-overlapping where possible, put the narrow chain first, and add tests for public, authenticated, and denied admin requests.

Interviewer: Would adding `@PreAuthorize` fix it?

Candidate: It gives defense in depth at the service boundary if method security is enabled and the call crosses the proxy, but it does not make the public request-chain mistake acceptable. I would fix both boundaries and keep the domain policy independent of the annotation.

2. Valid JWT, still 401

Interviewer: The token's signature is valid. Why reject it?

Candidate: Signature verifies integrity under a key, not suitability for this API. I still validate trusted issuer, intended audience, expiry and not-before with controlled clock skew, accepted algorithms, and any required tenant/client claims. I also inspect key rotation and authority mapping. I would not paste the raw production token into shared logs.

Interviewer: What is the 403 follow-up?

Candidate: If authentication succeeded but the principal lacks route or resource permission, authorization should deny it. I distinguish authentication entry-point behavior from access-denied handling and test both.

3. CSRF on a "REST API"

Interviewer: Can we disable CSRF because responses are JSON?

Candidate: Response format is irrelevant. I ask how the browser authenticates. If it automatically attaches a session cookie, cross-site requests can carry that credential and state changes need CSRF protection. If the API accepts only an explicit bearer header and has no cookie authentication, the CSRF threat differs. CORS is separate and is not authentication.

Interviewer: What if the SPA stores its bearer token in local storage?

Candidate: That avoids automatic cookie attachment but increases token theft impact under XSS. I would evaluate the complete browser threat model rather than trading one slogan for another.

4. Database outage restarts every pod

Interviewer: Kubernetes keeps restarting healthy Java processes when MySQL is down.

Candidate: The liveness probe likely includes database health. Liveness should answer whether the process is irrecoverably stuck, not whether every dependency is available. I would keep liveness local, use readiness or route-specific health to stop traffic that requires MySQL, keep diagnostic detail secured, and verify probe timeout/frequency so probes do not amplify the outage.

Interviewer: Should readiness always fail on database loss?

Candidate: Only if the instance cannot serve its promised routes. A service that can safely serve cached reads may remain ready for those paths, but a single platform readiness bit may force a conservative decision or require separated deployments/routes.

5. Retry a timed-out payment

Interviewer: Payment returned no response after 400 ms. Retry?

Candidate: Timeout is ambiguous: the provider may have committed. I retry only with a stable idempotency key and a provider contract that returns the original result, while respecting the remaining end-to-end deadline. Otherwise I query/reconcile before repeating the side effect. I record logical requests separately from attempts.

Interviewer: Add a circuit breaker too?

Candidate: Possibly, but only around classified dependency failure. It protects local and downstream capacity; it does not resolve the ambiguous operation. I would avoid multiplied gateway/client/mesh retries and make one layer own attempts.

6. WebFlux service is slower than MVC

Interviewer: We migrated controllers to `Mono`, but p99 became worse.

Candidate: A reactive return type does not make dependencies non-blocking. I would inspect event-loop thread dumps and latency spans for JDBC, filesystem, or blocking SDK calls. If the hot path remains blocking, MVC-possibly with a suitable execution model-may be simpler. If one legacy call must remain, I isolate it on a bounded scheduler and measure queueing; I do not scatter `subscribeOn` randomly.

Interviewer: Why not call `block()` once?

Candidate: On an event loop it consumes scarce progress capacity and can deadlock. Composition keeps the request asynchronous and preserves cancellation/context.

7. flatMap changed result order

Interviewer: A reactive export returns orders out of input order.

Candidate: `flatMap` subscribes to inner publishers concurrently and emits as they complete. If order is a requirement I use sequential `concatMap` or an ordered-concurrency strategy, with a bounded concurrency value. If order is not required, I change the test and API contract rather than paying for ordering accidentally.

Interviewer: Can concurrency be `Integer.MAX_VALUE`?

Candidate: That moves the queue into memory or the downstream. I size it against connection pools, downstream limits, response budget, and observed latency.

8. Batch restarts and duplicates

Interviewer: A failed job resumed from its checkpoint but duplicated 17 partner calls.

Candidate: The external side effects were not atomic with the batch metadata commit. Those calls likely succeeded before the chunk rolled back or the process died. I use a stable business key at the partner, an outbox/staging workflow, or reconciliation. I also test a crash exactly after the side effect and before checkpoint commit.

Interviewer: Does chunk size fix it?

Candidate: It changes the failure window and repeated work, not the atomicity across systems. Idempotency remains required.

9. Skip policy turns bad schema green

Interviewer: The job completed with 12,000 skipped rows after a vendor changed its file format.

Candidate: A schema change is systemic, not independent bad data. It should fail fast after header/schema validation or a low classified threshold. I reconcile input, committed, rejected, and quarantined counts and alert on deviation. Skip is acceptable only for explicitly tolerated records with an auditable disposition.

Interviewer: What would you expose operationally?

Candidate: Job/step identity, input checksum/version, counts by outcome/reason, duration, checkpoint/restart count, and the location of a protected reject artifact-without leaking sensitive rows into labels.

10. Integration flow loses errors

Interviewer: The sender got success, but an executor-backed handler failed later.

Candidate: The async channel moved failure to another thread, so it could not propagate normally to the sender. I define an error channel or adapter-specific retry/dead-letter path, correlate errors with the original message, bound executor and queue capacity, and state whether acceptance means "queued" or "completed."

Interviewer: Would a direct channel solve it?

Candidate: It restores synchronous error propagation but also makes the sender pay handler latency and failure. I choose it only if that coupling matches the contract.

11. Config refresh breaks latency

Interviewer: A config refresh changed read timeout from 500 ms to 2 s while retry stayed at three. What went wrong?

Candidate: Independently refreshable fields violated the total deadline. I bind a coherent, versioned resilience policy and validate attempts, per-attempt caps, backoff, and total budget together. Some client resources may not safely rebuild live, so I may require controlled rollout instead of refresh.

Interviewer: Roll back the property?

Candidate: Yes as incident mitigation, then add configuration validation, provenance/version telemetry, and a canary test so the unsafe combination cannot reach the fleet again.

12. Which Spring module?

Interviewer: We receive one 50 MB partner file each hour, transform it, and upload a result. Batch or Integration?

Candidate: I clarify restart, record count, checkpoint, protocols, and latency. If each file is a finite job requiring durable restart and item-level skip/retry, Batch should own processing. Integration can own SFTP polling, validation, and launching the job. If the file is small and one atomic transform is sufficient, a simpler scheduled service may be clearer.

Interviewer: Why not WebFlux?

Candidate: Non-blocking I/O can help a streaming transport, but it does not provide job identity, checkpoint, or restart semantics. I would not use it as a substitute for those requirements.

Answer rubric

Score each response from 0 to 2:

- identifies the invariant and asks one clarifying question;
- traces the actual framework/runtime boundary;
- names timeout, duplicate, context, restart, or ordering edge cases;
- chooses a bounded mechanism rather than "enable everything";
- proposes focused test and operational proof.

A score of 8/10 is a strong signal. Repeat any answer that starts only with an annotation name.

SDE-2 CORE CHAPTER

Chapter 9 - Practice, Reasoned Solutions, and Readiness

Attempt the exercises before reading the solutions. Each solution explains the decision boundary; production details still depend on the selected provider and deployment platform.

TRY IT | Exercises

1. Chain selection - Foundation

Three chains are declared in this order:

TEXT

```
A: /**          -> permitAll
B: /api/**      -> authenticated
C: /api/admin/** -> has ADMIN
```

Predict the policy for `GET /api/admin/users`. Repair the design.

2. Resource authorization - Interview Core

A caller has `order: refund` but belongs to tenant B. The order belongs to tenant A. Design the two authorization checks and state where each belongs.

3. Timeout arithmetic - Interview Core

An endpoint has a 1,000 ms deadline. Local work uses 120 ms and response reserve is 130 ms. Payment policy proposes three attempts with a 400 ms timeout and 50/100 ms backoff. Is it valid? Propose a bounded alternative.

4. Reactive diagnosis - Interview Core

This handler stalls under concurrency:

JAVA

```
Mono<Order> handle(String id) {
    return Mono.just(jdbcRepository.findById(id).orElseThrow());
}
```

Explain when the JDBC call runs and repair or replace the design.

5. Batch restart - SDE-2 Follow-up

A chunk writes 100 database rows, calls a partner once per row, and then commits batch metadata. The process dies after 70 partner calls. Design a safe restart strategy.

6. Integration error - Interview Core

An executor channel accepts a message; a later transformer throws. The HTTP caller already received 202. Define the error contract and evidence.

7. Observability labels - Foundation

Choose which values may be metric labels: HTTP method, normalized route, status family, order ID, raw exception message, dependency name, tenant ID.

8. Module selection - SDE-2 Follow-up

Choose the smallest approach for:

- 1 a 5,000-row nightly restartable import;
- 2 a blocking CRUD API with 40 requests/second;
- 3 a route that proxies 50,000 simultaneous slow streaming connections;
- 4 an SFTP-to-HTTP bridge with validation and dead-letter handling.

Solutions

Solution 1

The first matching chain wins, so A permits the admin request. Put narrow matchers before broad ones and avoid overlapping catch-all policies where possible:

TEXT

```
C: /api/admin/** -> ADMIN
B: /api/**       -> authenticated
A: explicitly public paths only -> permitAll
```

Test allowed and denied cases for each boundary; do not rely only on bean order inspection.

Solution 2

The request/method boundary may check the coarse `order:refund` authority. The application policy must then load the order and require `actor.tenantId == order.tenantId`, plus state/amount rules. A gateway cannot make the resource decision because it does not own authoritative order state. Test the policy in plain Java and one actual method/filter integration path.

Solution 3

The dependency budget is $1,000 - 120 - 130 = 750$ ms. Three 400 ms attempts plus 150 ms of backoff cannot fit. A valid policy might permit one 450 ms attempt and one retry capped by the remaining deadline, with 50 ms jittered backoff and reserve. Exact values need latency evidence. The operation also needs idempotency if it has side effects.

Solution 4

Java evaluates method arguments before calling `Mono.just`, so JDBC executes immediately on the caller thread. The code is not lazy or non-blocking. The clearest option is MVC for a blocking persistence stack. As a bounded migration bridge:

JAVA

```
return Mono.fromCallable(() -> jdbcRepository.findById(id).orElseThrow())
    .subscribeOn(Schedulers.boundedElastic());
```

This moves blocking work; it does not create more database connections or make JDBC asynchronous. Measure scheduler queue and database pool limits.

Solution 5

Database rows can share the chunk transaction only if the repository and metadata are correctly coordinated, but partner calls cannot. Give each logical row a stable operation key. Prefer committing an outbox/staging record with database state and let a separate idempotent dispatcher call the partner. On restart, already committed keys return the original result; pending keys resume. Reconcile partner responses with local status. Chunk size alone cannot close the crash window.

Solution 6

HTTP 202 must mean accepted for asynchronous processing, not completed. Persist or durably enqueue before returning if loss is unacceptable. Route transform failures to a correlated error/dead-letter store with retry classification, attempt cap, and alert. Measure accepted, completed, failed, retried, queue age, and rejections. An in-memory executor queue alone cannot survive process loss.

Solution 7

Good bounded labels: HTTP method, normalized route, status family, and a bounded dependency name. Order ID, raw exception text, and tenant ID can have unbounded cardinality and may be sensitive; put them in protected logs or sampled traces. If tenants are a tiny contractual set, a reviewed tenant tier-not raw tenant ID-may be a metric dimension.

Solution 8

- 1 Spring Batch if restart/checkpoint evidence matters; a simple scheduled transaction may suffice if all rows are one small atomic unit.
- 2 MVC with the blocking stack; WebFlux adds little.
- 3 WebFlux is a candidate if the entire I/O path is non-blocking and demand/cancellation are modeled.
- 4 Spring Integration is a candidate for adapters, routing, error flow, and dead-letter semantics; Batch may own processing if each file is a restartable finite job.

Final readiness assessment

You are ready to use these modules at SDE-2 level when you can:

- trace a servlet security request through the selected filter chain;
- separate authentication, coarse authorization, and resource policy;
- defend CSRF/CORS choices from the credential model;
- allocate one end-to-end deadline and keep retries idempotent;
- explain demand, subscription, scheduler, blocking, error, and cancellation behavior;
- distinguish job instance, execution, checkpoint, and idempotency;
- distinguish direct/pollable/executor channel failure semantics;
- design low-cardinality telemetry and useful probes;
- choose a smaller design when an extension adds no useful boundary.

Official sources and version checks

- [Spring Security servlet architecture](#)
- [Spring Security authorization](#)
- [Spring Cloud release-train compatibility](#)
- [Spring Framework WebFlux reference](#)
- [Project Reactor reference](#)
- [Spring Batch reference](#)
- [Spring Integration reference](#)
- [Spring Boot Actuator reference](#)

Before copying an API, confirm the exact reference documentation for the Boot-managed versions in the project. Concepts in this book are stable; names and defaults can move at major-version boundaries.

SDE-2 CORE CHAPTER

Chapter 10 - Java 21 Spring Ecosystem Runtime Companion

Executable models for Security chain selection, resource authorization, deadline-aware retries, reactive demand, batch idempotency, and bounded correlation state.

JAVA (continued 1/12)

```
import java.time.Duration;
import java.time.Instant;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.HashSet;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Objects;
import java.util.Optional;
import java.util.Set;
import java.util.function.Predicate;

/**
 * Dependency-free Java 21 checks for Spring ecosystem runtime reasoning.
 *
 * <p>This is deliberately not a replacement for Spring Security, Reactor,
 * Spring Cloud, Batch, or Integration. It makes the invariants hidden behind
 * those abstractions executable: first-match chain selection, resource-aware
 * authorization, remaining-deadline retries, demand, restart, idempotency, and
 * bounded correlation state.
 */
public final class SpringEcosystemExtensionsCompanion {
    private SpringEcosystemExtensionsCompanion() {}

    public static void main(String[] args) {
        validatesFirstMatchingSecurityChain();
        validatesResourceAwareAuthorization();
        validatesRemainingDeadlineRetryPlan();
        validatesDemandAccounting();
        validatesBatchRestartAndIdempotency();
        validatesCorrelationExpiry();
        System.out.println("PASS 6 Spring ecosystem extension invariant suites");
    }
}
```

JAVA (continued 2/12)

```

}

private static void validatesFirstMatchingSecurityChain() {
    List<SecurityChain> unsafeOrder = List.of(
        new SecurityChain("public-catch-all", request -> true, Decision.PERMIT),
        new SecurityChain("admin", request -> request.path().startsWith("/api/admin/"),
            Decision.REQUIRE_ADMIN));

    HttpRequest admin = new HttpRequest("GET", "/api/admin/users");
    require(select(unsafeOrder, admin).decision() == Decision.PERMIT,
        "first broad matcher must demonstrate the shadowing defect");

    List<SecurityChain> safeOrder = List.of(
        new SecurityChain("admin", request -> request.path().startsWith("/api/admin/"),
            Decision.REQUIRE_ADMIN),
        new SecurityChain("api", request -> request.path().startsWith("/api/"),
            Decision.AUTHENTICATED),
        new SecurityChain("public", request -> request.path().equals("/health"),
            Decision.PERMIT));

    require(select(safeOrder, admin).decision() == Decision.REQUIRE_ADMIN,
        "narrow admin chain must win");
    require(select(safeOrder, new HttpRequest("GET", "/api/orders/1")).decision()
        == Decision.AUTHENTICATED,
        "ordinary API path must require authentication");
    expectFailure(() -> select(safeOrder, new HttpRequest("GET", "/unknown")),
        "no security chain");
}

private static void validatesResourceAwareAuthorization() {
    Actor refundOperator = new Actor("user-7", "tenant-a", Set.of("order:refund"));
    Order owned = new Order("order-1", "tenant-a", OrderState.PAID, 9_000);
    Order foreign = new Order("order-2", "tenant-b", OrderState.PAID, 9_000);
    Order shipped = new Order("order-3", "tenant-a", OrderState.SHIPPED, 9_000);
}

```

JAVA (continued 3/12)

```
require(refundAllowed(refundOperator, owned, 2_000),
    "authorized actor should refund an eligible owned order");
require(!refundAllowed(refundOperator, foreign, 2_000),
    "authority must not cross tenant boundary");
require(!refundAllowed(refundOperator, shipped, 2_000),
    "resource state must be part of authorization");
require(!refundAllowed(refundOperator, owned, 10_000),
    "refund cannot exceed paid amount");
}

private static void validatesRemainingDeadlineRetryPlan() {
    RetryPolicy policy = new RetryPolicy(
        Duration.ofMillis(750),
        Duration.ofMillis(400),
        List.of(Duration.ofMillis(50), Duration.ofMillis(100)));

    List<AttemptBudget> plan = policy.plan();
    require(plan.size() == 2, "only two useful attempts fit in total budget");
    require(plan.get(0).timeout().equals(Duration.ofMillis(400)),
        "first attempt should receive configured cap");
    require(plan.get(1).timeout().equals(Duration.ofMillis(300)),
        "second attempt must use the remaining budget");
    require(plan.stream().map(AttemptBudget::timeout).reduce(Duration.ZERO, Duration::plus)
        .plus(Duration.ofMillis(50)).compareTo(Duration.ofMillis(750)) <= 0,
        "planned attempts and backoff must fit total deadline");

    expectFailure(() -> new RetryPolicy(
        Duration.ZERO, Duration.ofMillis(10), List.of()).plan(),
        "total budget");
}

private static void validatesDemandAccounting() {
    DemandWindow demand = new DemandWindow();
```

JAVA (continued 4/12)

```

        demand.request(3);
        require(demand.tryEmit("a"), "first demanded item should emit");
        require(demand.tryEmit("b"), "second demanded item should emit");
        require(demand.tryEmit("c"), "third demanded item should emit");
        require(!demand.tryEmit("overflow"), "publisher must not exceed demand");
        demand.request(1);
        demand.cancel();
        require(!demand.tryEmit("cancelled"), "cancelled subscriber must receive no item");
        require(demand.delivered().equals(List.of("a", "b", "c")),
            "delivery history should be deterministic");
    }

    private static void validatesBatchRestartAndIdempotency() {
        IdempotentPartner partner = new IdempotentPartner();
        BatchCheckpoint checkpoint = new BatchCheckpoint();
        List<String> itemKeys = List.of("settlement-1", "settlement-2", "settlement-3");

        processUntilCrash(itemKeys, checkpoint, partner, 2);
        require(checkpoint.lastCommittedIndex() == 0,
            "crash before chunk commit leaves checkpoint unchanged");
        require(partner.physicalEffects() == 2,
            "two remote effects happened before crash");

        processAndCommit(itemKeys, checkpoint, partner);
        require(checkpoint.lastCommittedIndex() == 3,
            "restart must advance checkpoint after successful chunk");
        require(partner.physicalEffects() == 3,
            "stable keys must suppress duplicate physical effects");
        require(partner.requests() == 5,
            "restart can repeat requests even when effects are deduplicated");
    }

    private static void validatesCorrelationExpiry() {
        Instant start = Instant.parse("2026-08-01T00:00:00Z");
    }

```

JAVA (continued 5/12)

```
CorrelationStore store = new CorrelationStore(Duration.ofMinutes(5));
store.add("group-a", "part-1", start);
store.add("group-b", "part-1", start.plusSeconds(60));
store.add("group-a", "part-2", start.plusSeconds(120));

require(store.parts("group-a").equals(List.of("part-1", "part-2")),
    "correlation group must preserve insertion order");
int removed = store.expireBefore(start.plusSeconds(330));
require(removed == 1, "only stale group-a should expire");
require(store.parts("group-a").isEmpty(), "expired group must be removed");
require(store.parts("group-b").equals(List.of("part-1")),
    "fresh group must remain");
}

static SecurityChain select(List<SecurityChain> chains, HttpRequest request) {
    return chains.stream()
        .filter(chain -> chain.matches().test(request))
        .findFirst()
        .orElseThrow(() -> new IllegalArgumentException(
            "no security chain for " + request.path()));
}

static boolean refundAllowed(Actor actor, Order order, long amountInCents) {
    Objects.requireNonNull(actor, "actor");
    Objects.requireNonNull(order, "order");
    return actor.authorities().contains("order:refund")
        && actor.tenantId().equals(order.tenantId())
        && order.state() == OrderState.PAID
        && amountInCents > 0
        && amountInCents <= order.paidInCents();
}

private static void processUntilCrash(
```


JAVA (continued 6/12)

```
        List<String> keys,
        BatchCheckpoint checkpoint,
        IdempotentPartner partner,
        int crashAfterRequests) {
    for (int index = checkpoint.lastCommittedIndex(); index < keys.size(); index++) {
        partner.apply(keys.get(index));
        if (partner.requests() == crashAfterRequests) {
            return;
        }
    }
}

private static void processAndCommit(
    List<String> keys,
    BatchCheckpoint checkpoint,
    IdempotentPartner partner) {
    for (int index = checkpoint.lastCommittedIndex(); index < keys.size(); index++) {
        partner.apply(keys.get(index));
    }
    checkpoint.commit(keys.size());
}

private static void expectFailure(Runnable action, String expectedText) {
    try {
        action.run();
        throw new AssertionError("expected failure containing: " + expectedText);
    } catch (IllegalArgumentException failure) {
        require(failure.getMessage().contains(expectedText),
            "unexpected failure: " + failure.getMessage());
    }
}

private static void require(boolean condition, String message) {
```

JAVA (continued 7/12)

```
        if (!condition) {
            throw new AssertionError(message);
        }
    }

    record HttpRequest(String method, String path) {
        HttpRequest {
            Objects.requireNonNull(method, "method");
            Objects.requireNonNull(path, "path");
        }
    }

    record SecurityChain(String name, Predicate<HttpRequest> matches, Decision decision) {
        SecurityChain {
            Objects.requireNonNull(name, "name");
            Objects.requireNonNull(matches, "matches");
            Objects.requireNonNull(decision, "decision");
        }
    }

    enum Decision {
        PERMIT,
        AUTHENTICATED,
        REQUIRE_ADMIN
    }

    record Actor(String id, String tenantId, Set<String> authorities) {
        Actor {
            Objects.requireNonNull(id, "id");
            Objects.requireNonNull(tenantId, "tenantId");
            authorities = Set.copyOf(authorities);
        }
    }
}
```

JAVA (continued 8/12)

```
record Order(String id, String tenantId, OrderState state, long paidInCents) {
    Order {
        Objects.requireNonNull(id, "id");
        Objects.requireNonNull(tenantId, "tenantId");
        Objects.requireNonNull(state, "state");
    }
}

enum OrderState {
    PAID,
    SHIPPED,
    CANCELLED
}

record AttemptBudget(int attempt, Duration timeout) {
    AttemptBudget {
        if (attempt < 1 || timeout.isNegative() || timeout.isZero()) {
            throw new IllegalArgumentException("invalid attempt budget");
        }
    }
}

record RetryPolicy(Duration totalBudget, Duration perAttemptCap, List<Duration> backoffs) {
    RetryPolicy {
        Objects.requireNonNull(totalBudget, "totalBudget");
        Objects.requireNonNull(perAttemptCap, "perAttemptCap");
        backoffs = List.copyOf(backoffs);
    }

    List<AttemptBudget> plan() {
        if (totalBudget.isNegative() || totalBudget.isZero()) {
            throw new IllegalArgumentException("total budget must be positive");
        }
    }
}
```

JAVA (continued 9/12)

```

    }
    if (perAttemptCap.isNegative() || perAttemptCap.isZero()) {
        throw new IllegalArgumentException("per-attempt cap must be positive");
    }

    List<AttemptBudget> result = new ArrayList<>();
    Duration remaining = totalBudget;
    int attempt = 1;
    while (!remaining.isZero() && !remaining.isNegative()) {
        Duration timeout = remaining.compareTo(perAttemptCap) < 0
            ? remaining : perAttemptCap;
        result.add(new AttemptBudget(attempt, timeout));
        int backoffIndex = attempt - 1;
        if (backoffIndex >= backoffs.size()) {
            break;
        }
        Duration backoff = backoffs.get(backoffIndex);
        remaining = remaining.minus(timeout).minus(backoff);
        attempt++;
    }
    return List.copyOf(result);
}

static final class DemandWindow {
    private long outstanding;
    private boolean cancelled;
    private final List<String> delivered = new ArrayList<>();

    void request(long count) {
        if (count <= 0) {
            throw new IllegalArgumentException("demand must be positive");
        }
    }
}

```

JAVA (continued 10/12)

```
        outstanding = Math.addExact(outstanding, count);
    }

    boolean tryEmit(String item) {
        Objects.requireNonNull(item, "item");
        if (cancelled || outstanding == 0) {
            return false;
        }
        outstanding--;
        delivered.add(item);
        return true;
    }

    void cancel() {
        cancelled = true;
    }

    List<String> delivered() {
        return List.copyOf(delivered);
    }
}

static final class BatchCheckpoint {
    private int lastCommittedIndex;

    int lastCommittedIndex() {
        return lastCommittedIndex;
    }

    void commit(int nextIndex) {
        if (nextIndex < lastCommittedIndex) {
            throw new IllegalArgumentException("checkpoint cannot move backward");
        }
    }
}
```

JAVA (continued 11/12)

```
        lastCommittedIndex = nextIndex;
    }
}

static final class IdempotentPartner {
    private final Set<String> completedKeys = new HashSet<>();
    private int requests;
    private int physicalEffects;

    String apply(String key) {
        Objects.requireNonNull(key, "key");
        requests++;
        if (completedKeys.add(key)) {
            physicalEffects++;
        }
        return "receipt:" + key;
    }

    int requests() {
        return requests;
    }

    int physicalEffects() {
        return physicalEffects;
    }
}

static final class CorrelationStore {
    private final Duration ttl;
    private final Map<String, Group> groups = new LinkedHashMap<>();

    CorrelationStore(Duration ttl) {
        if (ttl.isNegative() || ttl.isZero()) {
```

JAVA (continued 12/12)

```

        throw new IllegalArgumentException("ttl must be positive");
    }
    this.ttl = ttl;
}

void add(String key, String part, Instant now) {
    Objects.requireNonNull(key, "key");
    Objects.requireNonNull(part, "part");
    Objects.requireNonNull(now, "now");
    Group group = groups.computeIfAbsent(key,
        ignored -> new Group(now, new ArrayList<>()));
    group.parts().add(part);
}

int expireBefore(Instant now) {
    List<String> expired = groups.entrySet().stream()
        .filter(entry -> !entry.getValue().createdAt().plus(ttl).isAfter(now))
        .map(Map.Entry::getKey)
        .toList();
    expired.forEach(groups::remove);
    return expired.size();
}

List<String> parts(String key) {
    return Optional.ofNullable(groups.get(key))
        .map(group -> List.copyOf(group.parts()))
        .orElseGet(List::of);
}

private record Group(Instant createdAt, List<String> parts) {
}
}

```

Part II - Practice and Handoff

TRY IT | Practice Ladder and Next Steps

TRY IT | Practice ladder

- Foundation: module purpose and selection
- Interview Core: Security, Actuator, testing, and Cloud
- SDE-2 Follow-up: reactive, batch, and integration trade-offs

For every coding problem, state the input contract, select the numeric and collection types deliberately, write the invariant before the loop or recursion, test the smallest and largest valid inputs, and close with time and auxiliary-space complexity.

Navigation

[Previous book: FW 10 - Apache Kafka and Spring Kafka](#)

[Series index](#)

[Next book: FW 12 - Spring AI for Java Engineers](#)

TRY IT | Completion check

- ☐ I can recognize the core patterns without being told the data structure or algorithm.
- ☐ I can implement the representative Java templates from memory.
- ☐ I can explain why each implementation is correct.
- ☐ I can test boundaries, invalid inputs, overflow, and adversarial shapes.
- ☐ I can answer at least one SDE-2 follow-up involving trade-offs or production constraints.

Series Roadmap

Choose Java, DSA, Frameworks, or System Design first, then follow the books inside that segment in order. The same series codes and positions appear on the website, PDF covers, and canonical download folders.

SEGMENT	BOOK	FOCUSED LEARNING PATH
Java	JAVA 01	Java Foundations for Problem Solving
Java	JAVA 02	Git and GitHub for Java Engineers
Java	JAVA 03	Maven and Gradle for Java Engineers
Java	JAVA 04	Language, OOP, and Modern Java
Java	JAVA 05	Collections, Streams, and I/O
Java	JAVA 06	JVM and Java Execution
Java	JAVA 07	Java Concurrency and the Memory Model
Java	JAVA 08	Performance, Diagnostics, and GC Incidents
Java	JAVA 09	Java Question Bank, Study Plan, and Reference
DSA	DSA 01	Time and Space Complexity for Java Interviews
DSA	DSA 02	Number Systems and Math Foundations for DSA Interviews
DSA	DSA 03	Number Systems Interview Patterns and Rapid Revision
DSA	DSA 04	Bit Manipulation in Java
DSA	DSA 05	Loop Mastery, Patterns, and Index Calculations
DSA	DSA 06	Arrays and Array Problem-Solving Patterns
DSA	DSA 07	Strings and String Problem-Solving Patterns
DSA	DSA 08	Hashing: Maps, Sets, Frequency, and Prefix State
DSA	DSA 09	Recursion and Backtracking in Java
DSA	DSA 10	Linked Lists: Pointer Reasoning and Mutation
DSA	DSA 11	Stacks, Queues, Deques, and Monotonic Patterns
DSA	DSA 12	Binary Search: Bounds, Answers, and Invariants
DSA	DSA 13	Trees, Binary Search Trees, and Tries
DSA	DSA 14	Heaps, Priority Queues, Selection, and Top-K
DSA	DSA 15	Graphs: Traversal, Ordering, Paths, and Union-Find
DSA	DSA 16	Greedy Algorithms: Recognition, Proofs, and Scheduling
DSA	DSA 17	Dynamic Programming: State, Transitions, and Optimization
Frameworks	FW 01	MySQL for Java Backend Interviews
Frameworks	FW 02	Hibernate and JPA for Java Backend Interviews
Frameworks	FW 03	Spring Framework for Java Engineers
Frameworks	FW 04	Spring Boot for Java Backend Engineers
Frameworks	FW 05	Spring Boot and REST APIs

SEGMENT	BOOK	FOCUSED LEARNING PATH
Frameworks	FW 06	Spring Data for Java Backend Engineers
Frameworks	FW 07	MongoDB for Java Backend Interviews
Frameworks	FW 08	Redis for Java Backend Interviews
Frameworks	FW 09	Persistence, SQL, JPA, and Caching
Frameworks	FW 10	Apache Kafka and Spring Kafka
Frameworks	FW 11	Spring Ecosystem Extensions
Frameworks	FW 12	Spring AI for Java Engineers
System Design	SD 01	Design, Backend, Testing, and Security
System Design	SD 02	Distributed Systems and System Design

All 40 publication editions are available now. Choose Java, DSA, Frameworks, or System Design first, then follow the books inside that shelf in order. Each deep-dive book remains independently printable and available on the web.

About the Author

Vinay Reddy Kalluri

Vinay Reddy Kalluri is a senior Java backend engineer and the author and editor of the Java SDE-2 Interview Preparation Series. His work spans high-throughput microservices, event-driven architecture, resilient APIs, large-scale data processing, cloud delivery, and production performance across healthcare and enterprise systems.

Editorial approach

Vinay sets the learning sequence and publication standard and performs the final review for Java accuracy, evidence, attribution, and release readiness. Individual contributors retain visible credit for accepted original work through AUTHORS.md and the public Git history.

What distinguishes his approach is the combination of hands-on system design and operational ownership. He treats timeouts, retries, idempotency, dead-letter recovery, data consistency, SQL behavior, caching, and observability as part of the design rather than afterthoughts. He has also mentored engineers and led modernization, migration, and reliability work across distributed services.

Selected engineering and academic highlights

- More than eight years building Java and Spring Boot services for high-throughput healthcare and enterprise platforms.
- Designed Kafka-based event systems that have processed more than one million events per hour, with explicit idempotency, retry, recovery, and observability controls.
- M.S. in Computer Science from the University of Missouri-Kansas City (3.9/4.0) and M.S. in Software Engineering from VIT University (9.3/10), where he was a Gold Medalist.
- Independent builder of Work Visa Insights, an immigration-data intelligence platform, and Play Together, a published Android application.

Why this series exists

Vinay created this series to turn fragmented interview preparation into a deliberate progression: learn the fundamentals, run the examples, predict behavior, debug failures, explain trade-offs, and only then move into advanced engineering. The books reflect the same principle he applies to production systems: clarity before cleverness, explicit contracts, measurable behavior, and reliable foundations.

Connect: [LinkedIn](#) | [GitHub](#)

Copyright and Publishing Notes

Copyright 2026 Vinay Reddy Kalluri and credited contributors. Open educational edition.

Book prose, exercises, diagrams, and PDFs are licensed under Creative Commons Attribution 4.0 International (CC BY 4.0). Build scripts and source code are licensed under MIT. Individual credit is recorded in AUTHORS.md and Git history.

Repository: <https://github.com/vinayreddykalluri/SDE2-Interview-Handbook>. Attribution does not imply endorsement. Java and related marks are owned by their respective holders.

Examples target Java 21 unless a section states otherwise. Validate release-specific APIs, JVM behavior, security, performance, licensing, and operational requirements before production use.

Series position: Frameworks, Data, and Messaging - Book 11 of 12 - Study Step FW-11. The local table of contents and PDF bookmarks navigate within this file; the roadmap and printed filenames navigate across the complete series.

Source provenance: curated from the independent Java SDE-2 master guide plus focused original material. Original master chapter numbers are labeled explicitly when retained in cross-references.